A Transformer-Based Conversational Agent for Ubuntu Technical Support(Reinforcement-bot)

Group Name: The Salamanca Squad

Manoj Kumar Surabhi
M.S. Data Science
mksurabh@mtu.edu

Sumanth Reddy Thandra
M.S. Data Science
thandra@mtu.edu

Rahul Sajith
M.S. Data Science
sajith@mtu.edu

***Abstract*—The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best-performing models also connect the encoder and decoder through an attention mechanism. We adopt a new, simplified network architecture—the Transformer—based solely on attention mechanisms, dispensing with recurrence and convolutions entirely.**

In this project, we train a Transformer-based chatbot using the Ubuntu Dialogue Corpus for technical support conversations. The dataset comprises over 36.9 million dialogue lines, 373 million words, and 4 billion characters from three CSV files. The large and diverse dataset helps the model learn technical dialogue context effectively.

Our model achieved a training loss of 4.236, validation loss of 4.027, and a BLEU score on the Ubuntu test data, indicating strong response generation quality. The Transformer generalizes well to varied technical conversations while training significantly faster than traditional models, making it a solid candidate for broader NLP tasks.

I. Introduction

The rapid growth in demand for automated technical support systems has led to significant advances in the development of conversational agents. Among these, transformer-based architectures have emerged as state-of-the-art models capable of understanding and generating human-like responses. This project presents the design and implementation of a transformer-based chatbot tailored for Ubuntu-related technical support, leveraging the extensive Ubuntu Dialogue Corpus.

The objective of this work is to create a context-aware dialogue system that can engage in multi-turn conversations and provide accurate, relevant responses to user queries. Unlike traditional retrieval-based or rule-based systems, this model is built using an encoder-decoder transformer framework, enabling it to learn nuanced representations of input dialogues and generate meaningful replies.

To enhance model performance, we employed a thorough data preprocessing pipeline that involved merging multiple Ubuntu dialogue datasets, cleaning noise (e.g., URLs, HTML, special characters), filtering incomplete dialogues, and normalizing text for efficient tokenization. A subword tokenizer was trained from scratch using SentencePiece to handle rare words and ensure vocabulary flexibility.

The final architecture integrates positional encoding, multi-head self-attention, encoder-decoder attention mechanisms, and greedy decoding. The system outputs fluent and relevant responses, demonstrating its effectiveness in a technical support domain. This work contributes a scalable, trainable, and domain-specific chatbot architecture suitable for deployment in real-world support environments.

II. Related Works

Early dialogue systems primarily relied on rule-based or retrieval-based techniques, which used predefined response templates or matched inputs against a fixed database. These methods were limited in scalability and failed to handle open-ended or context-rich conversations effectively.

The evolution toward statistical and neural approaches led to the adoption of sequence-to-sequence models using recurrent neural networks. Variants such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) improved the handling of long-range dependencies in dialogue but suffered from limitations in parallelization and performance on longer sequences.

The introduction of the Transformer architecture marked a major breakthrough in natural language processing. Unlike RNN-based models, Transformers rely entirely on attention mechanisms, allowing models to process entire sequences in parallel and learn more complex dependencies. This architecture has since become foundational for numerous language understanding and generation models.

In domain-specific applications like technical support, dialogue datasets have been curated to train and benchmark conversational agents. Among them, the Ubuntu Dialogue Corpus is widely used for multi-turn conversation modeling, offering real-world scenarios involving Linux-based troubleshooting discussions.

Building upon this foundation, our project leverages a Transformer-based encoder-decoder model to generate contextually relevant responses in Ubuntu-related support dialogues. By focusing on generative modeling rather than response retrieval, this approach enables more flexible and adaptive interactions within the chatbot framework.

III. Data Description

The dataset used in this project is derived from three structured dialogue sources focused on Ubuntu-based technical support conversations. These datasets were combined, cleaned, and formatted to train a context-aware conversational model.

A. Size of Data

The data originated from three separate dialogue datasets: `dialogueText`, `dialogueText_196`, and

`dialogueText_301`. After merging and preprocessing, the final corpus included:

- Total raw messages: 165,429,258
- Final cleaned messages (after filtering): ~165 million
- Final dialogues (multi-turn only): 1,837,770
- Columns per entry: 6
  `folder, dialogueID, date, from, to, text`

Additional preprocessing included removal of rows with null or empty text, filtering of dialogues with only a single message turn, and exclusion of invalid one-sided conversations.

B. Categorical or Continuous Variables

The dataset primarily consists of categorical and textual data, with no traditional numerical predictors. The key variables are:

- **Categorical:**
  - `folder` – indicates source directory or data partition
  - `dialogueID` – unique identifier for each dialogue
  - `from`, `to` – speaker and recipient identifiers
  - `date` – timestamp as a categorical string (used for ordering but not modeled directly)
- **Textual:**
  - `text` – the raw dialogue message content (subjected to cleaning, tokenization, and embedding)
- No continuous variables (e.g., numerical scores or sensor readings) are present, as the dataset is focused entirely on unstructured human conversation.

C. Distribution of Predictor Variables

The primary predictor variable for the model is the **cleaned text**, which is converted into subword tokens during preprocessing. Token length (message length in words) distribution was analyzed:

- **Mean token count per message:** ~10
- **Minimum tokens (after cleaning):** 1
- **Maximum tokens:** 198
- **Distribution shape:** Right-skewed
  - Most messages fall in the 5–15 token range
  - A smaller fraction contains verbose technical instructions or logs
- This insight was critical in selecting an appropriate **maximum sequence length** for model input (e.g., 30 tokens)

IV. Method

A. Data Pre-processing

Effective data pre-processing is a critical step in building reliable natural language understanding systems, especially in dialogue modeling where inconsistencies, noise, and structural irregularities are common. For this project, multiple stages of cleaning, normalization, and transformation were applied to ensure high-quality input for the Transformer model.

B. Data Integration and De-duplication

The initial dataset was compiled by merging three separate sources: `dialogueText`, `dialogueText_196`, and `dialogueText_301`. Each source contained structured dialogues represented through metadata fields: *folder*, *dialogueID*, *date*, *from*, *to*, and *text*.

To ensure uniqueness, a composite key was formed using *folder* and *dialogueID*. Duplicate message rows across the datasets were identified and removed during the merge process.

	folder	dialogueID	date	from	to	text
0	3	126125.tsv	2008-04-23 14:55:00	bad_image	NaN	Hello folks, please help me a bit with the following sentence: 'Order here your personal photos or videos.' - I think the only allowed version is 'Order your personal videos or photos here.', but I'm not sure, are you?

Fig. 1: Combined data text output

C. Null and Incomplete Data Handling

Null value inspection revealed missing entries in several fields:

- **from:** 287 rows
- **to:** approximately 5.7 million rows
- **text:** 880 rows

Since the *text* field is essential for model training, rows with missing or empty text were dropped. The *from* and *to* fields were preserved even with missing values, as certain system prompts or initial messages may not contain sender or receiver information.

D. Dialogue Validation and Filtering

Dialogues were grouped using *folder* and *dialogueID*. Interaction quality was assessed based on:

- Number of unique speakers (*from*)
- Number of unique recipients (*to*)
- Message time intervals

Dialogues involving only one unique participant were considered invalid and removed, filtering out incomplete or one-sided exchanges. This process reduced the dataset by only 0.43%.

E. Token Count Analysis and Empty Row Removal

Each message was tokenized to compute its word count (`text_token_count`). Summary statistics showed:

- Mean tokens per message: approximately 10
- Maximum: 198
- Minimum: 0

Messages with a token count of zero (i.e., blank or whitespace-only messages) were removed, eliminating 1,252 rows.

F. Text Cleaning and Normalization

A custom cleaning function was applied to all text entries to reduce noise and standardize formatting. The process included:

- **Noise Removal:** URLs, HTML tags, special characters, underscores, and hyphens
- **Normalization:**

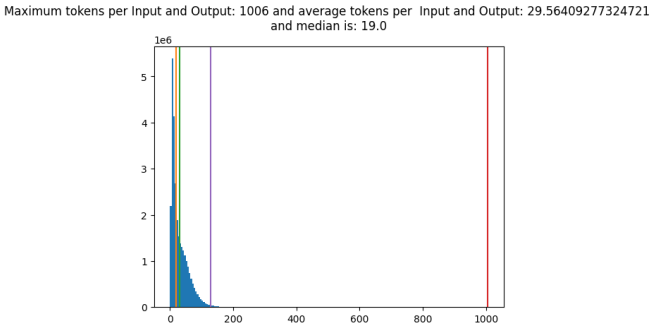

Fig. 2: Distribution of Tokens per Input and Output in the Dataset

- Conversion to lowercase
- Reduction of elongated characters (e.g., `soooo` to `so`)
- Collapsing multiple whitespaces into a single space

This step significantly improved vocabulary consistency for downstream tokenization.

G. Token Statistics Recalculation

Token lengths were recalculated after cleaning:

- New minimum token count: 1
- Maximum token count remained: 198

This verified the effectiveness of the cleaning pipeline and ensured reliable sequence length configuration.

H. Dialogue Structuring and Final Filtering

Messages were re-grouped into dialogues using the `dialogue_texts` list format. Dialogues with only a single message turn were excluded, removing 1,159 additional entries. This ensured that the final set reflected meaningful multi-turn interactions.

I. Final Outputs for Modeling

Two key outputs were prepared:

- **cleaned_dialogues_corpus.csv:** Contains structured multi-turn dialogues with metadata and cleaned text
- **full.txt:** A plain text file where each line is a cleaned individual message, used for training a tokenizer

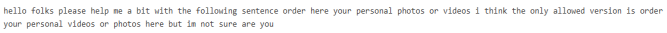
hello folks please help me a bit with the following sentence order here your personal photos or videos i think the only allowed version is order your personal videos or photos here but im not sure are you

Fig. 3: Cleaned Corpus data text

Special tokens such as `<BOS>` and `<EOS>` were later added to facilitate sequence boundary detection during model training and inference.

V. Feature Extraction

Feature extraction plays a pivotal role in transforming raw textual input into meaningful representations that machine learning models can process. For this project, feature extraction involved tokenization, subword modeling, embedding generation, and positional encoding — all of which serve as critical inputs to the Transformer-based architecture.

A. Subword Tokenization

Given the domain-specific vocabulary of the Ubuntu Dialogue Corpus, a subword tokenization strategy was adopted to effectively handle rare, compound, and out-of-vocabulary terms. The SentencePiece tokenizer was trained on the full corpus (`full.txt`) using a unigram language model.

This method splits words into smaller subword units, allowing the model to generalize across similar terms and improve robustness to spelling variations and unseen words.

B. Special Token Addition

To guide the sequence generation process, each message was wrapped with special tokens:

- `<BOS>` – Beginning-of-sentence token
- `<EOS>` – End-of-sentence token

These tokens are essential for defining the boundaries of each sequence during training and inference in the encoder-decoder framework.

C. Word Embeddings

After tokenization, each subword token is transformed into a dense vector using an embedding layer. These embeddings encode semantic relationships between tokens by mapping them into a continuous vector space.

The embedding dimension (`d_model`) was chosen to ensure a balance between representation quality and computational efficiency. These vectors are trainable and updated during the learning process.

D. Positional Encoding

Transformers do not inherently encode sequence order. To address this, sinusoidal positional encodings were added to each token embedding. These encodings assign unique position-based vectors using sine and cosine functions of varying frequencies.

The result is that each token representation carries both semantic meaning and positional context, which is essential for modeling word order in conversations.

E. Final Input Representation

The final input to the Transformer encoder is the sum of the token embeddings and positional encodings. This enriched representation is passed through multiple layers of attention and feed-forward operations, allowing the model to learn deep contextual relationships between tokens within each message.

This feature extraction pipeline ensures that the model receives input that is both syntactically and semantically informed, facilitating accurate understanding and generation of dialogue.

Hyperparameter Tuning and Model Selection

Selecting an appropriate model architecture and optimizing its hyperparameters are essential to achieving high performance in sequence-to-sequence tasks. In this project, we focused on tuning key parameters in the Transformer encoder-decoder architecture to enhance generation quality while maintaining computational efficiency.

A. Model Architecture Selection

Most competitive neural sequence transduction models have an encoder-decoder structure. Here, the encoder maps an input sequence of symbol representations (x1, ..., xn) to a sequence of continuous representations z = (z1, ..., zn). Given z, the decoder then generates an output sequence (y1, ..., ym) of symbols one element at a time. At each step the model is auto-regressive , consuming the previously generated symbols as additional input when generating the next. The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

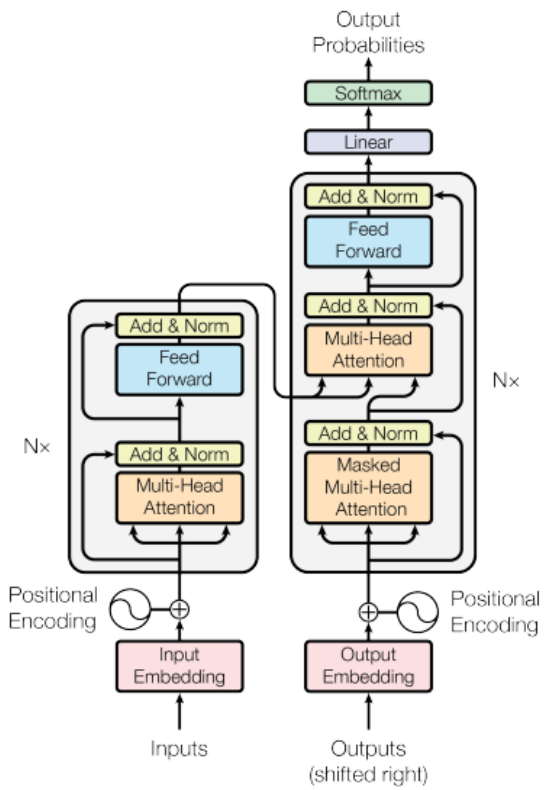

Figure 1: The Transformer - model architecture.

Fig. 4: Transformer Encoder-Decoder Architecture

Encoder: The encoder is composed of a stack of N identical layers. Each layer contains two sub-layers. The first is a multi-head self-attention mechanism, and the second is a position-wise, fully connected feed-forward network. A residual connection is employed around each of the two sub-layers, followed by layer normalization. Specifically, the output of each sub-layer is computed as:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

where `Sublayer(x)` represents the function implemented by the sub-layer itself. To support these residual connections, all sub-layers in the model, along with the embedding layers, produce outputs of a fixed dimension d_{model}.

Decoder: The decoder is also composed of a stack of N identical layers. In addition to the two sub-layers in each encoder layer, the decoder includes a third sub-layer that performs multi-head attention over the output of the encoder stack. As in the encoder, residual connections and layer normalization are applied around each of the sub-layers.

To preserve the auto-regressive nature of generation, the self-attention sub-layer in the decoder is modified to prevent positions from attending to subsequent positions. This is achieved through masking, which ensures that the prediction for position i depends only on known outputs at positions less than i, as the output embeddings are offset by one position.

F. Attention Mechanism

An attention function maps a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is derived from a compatibility function between the query and the corresponding key.

1) Scaled Dot-Product Attention: The attention mechanism used in this work is called *Scaled Dot-Product Attention*. The inputs consist of queries and keys of dimension d_k, and values of dimension d_v. The attention is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

In practice, the attention function is computed on a set of queries simultaneously, with queries Q, keys K, and values V packed into matrices. The dot product of Q and K^T is scaled by $\sqrt{d_k}$ to avoid excessively large values that would push the softmax into regions with very small gradients.

Dot-product attention is computationally more efficient than additive attention and can leverage highly optimized matrix multiplication. Although additive attention can outperform unscaled dot-product attention for large d_k, the scaling factor in scaled dot-product attention mitigates this limitation.

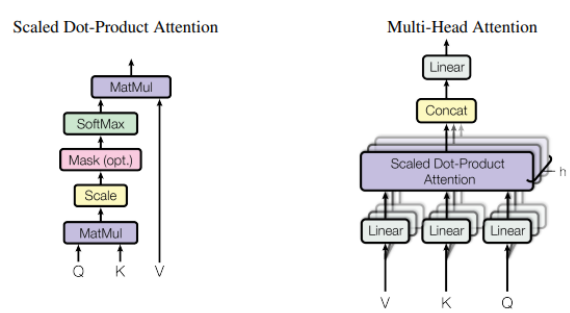

Fig. 5: Attention

2) Multi-Head Attention: Rather than applying a single attention function, the Transformer model uses *Multi-Head Attention*, which linearly projects the queries, keys, and values h times using different learned projections. These are then processed in parallel, and the outputs are concatenated and projected again.

The multi-head attention formula is:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

$$\text{where} \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Here, the projection matrices are defined as:

$$W_i^Q \in R^{d_{\text{model}} \times d_k}, \quad W_i^K \in R^{d_{\text{model}} \times d_k}, \quad W_i^V \in \times d_{\text{model}}$$

In this implementation, $h = 8$ attention heads are used, with each head using:

$$d_k = d_v = \frac{d_{\text{model}}}{h} = 64$$

This design enables the model to attend to information from multiple representation subspaces simultaneously. Despite using multiple heads, the total computational cost is comparable to that of a single-head attention mechanism operating at full dimensionality.

G. Position-wise Feed-Forward Networks

In addition to attention sub-layers, each layer in both the encoder and decoder contains a fully connected feed-forward network, which is applied to each position independently and identically. This feed-forward network consists of two linear transformations with a ReLU activation function in between. It is defined as:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Although the transformations are the same across all positions, each layer uses its own set of parameters. This operation can also be interpreted as applying two one-dimensional convolutions with a kernel size of 1. The input and output dimensionality is $d_{\text{model}} = 512$, and the inner-layer dimensionality is $d_{ff} = 2048$.

H. Embeddings and Softmax

Similar to other sequence transduction models, learned embeddings are used to convert input and output tokens into continuous vectors of dimension d_{model}. The decoder output is then passed through a learned linear transformation and a softmax layer to produce probabilities over the vocabulary for the next predicted token.

In this architecture, the same weight matrix is shared between the input embedding, output embedding, and the pre-softmax linear transformation. To stabilize the scale of the embeddings, the weight vectors are multiplied by $\sqrt{d_{\text{model}}}$ during training.

I. Positional Encoding

Since the Transformer model does not use recurrence or convolution, it lacks an inherent mechanism for capturing the order of sequences. To address this, positional encodings are added to the input embeddings at the base of both the encoder and decoder stacks.

These encodings have the same dimension as the embeddings (d_{model}), allowing the two to be summed directly. In this implementation, fixed positional encodings are generated using sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Here, pos denotes the position and i represents the dimension. Each dimension of the positional encoding corresponds to a sinusoid, with wavelengths forming a geometric progression from 2π to $10000 \cdot 2\pi$.

This encoding strategy enables the model to attend by relative position and allows for generalization to sequence lengths not seen during training. While learned positional embeddings were also tested, the fixed sinusoidal encoding yielded comparable results and was chosen for its ability to support extrapolation.

B. Key Hyperparameters Tuned

The following hyperparameters were tuned through iterative experimentation and evaluation on validation data:

- **Embedding Dimension (d_model):** 256
  Controls the size of word and positional embeddings, balancing model expressiveness and memory usage.
- **Number of Attention Heads (nhead):** 8
  Allows the model to attend to different subspaces of token relationships.
- **Number of Encoder/Decoder Layers (num_layers):** 3
  More layers improve model depth; too many can lead to overfitting and longer training.
- **Feedforward Network Size (ff_dim):** 1024
  Intermediate size in feed-forward submodules.
- **Dropout Rate:** 0.1
  Helps prevent overfitting.
- **Maximum Sequence Length:** 30 tokens
  Based on token distribution analysis.
- **Vocabulary Size:** 10,000 subword units
  Determined using SentencePiece tokenizer.
- **Batch Size:** 64
  Tuned to fit available GPU memory.
- **Learning Rate:** Custom schedule with warm-up steps for early stabilization.

C. Optimization Strategy

The model was trained using the Adam optimizer with the following configuration:

- $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$
- Linear warm-up for the first 4,000 steps
- Cross-entropy loss used, with padding tokens masked out

VI. Experiment and Results

subsectionTraining and Validation Loss

The Transformer-based dialogue model was trained on approximately 200,000 batches generated from the Ubuntu Dialogue Corpus. The training loss decreased consistently over the course of the epoch, beginning at **4.707** and reaching **4.236** after 200,000 batches. Validation was performed using a separate 24% subset of the dataset, resulting in a **final validation loss of 4.027**.

The fact that the validation loss is slightly lower than the training loss suggests the model maintained good generalization capabilities and did not overfit during training. The consistent loss reduction across batches indicates that the model effectively learned from the training data.

A. Evaluation Metrics

The model's output was evaluated using the Bilingual Evaluation Understudy (BLEU) metric, which measures n-gram precision by comparing model-generated responses to human-authored references. After evaluating 20 batches from the test set, the model achieved a **BLEU score of 1.034**.

This relatively low BLEU score reflects the early stage of training and the complexity of open-domain dialogue tasks, where multiple valid responses may exist for a given query. Moreover, BLEU is sensitive to surface-level variations and may not fully capture semantic appropriateness, especially when used over a small test sample.

B. Summary of Results

TABLE I: Training, Validation, and Evaluation Results

Metric	Value	Description
Initial Training Loss	4.707	Starting point of training loss
Final Training Loss	4.236	Loss after one full training epoch
Final Validation Loss	4.027	Indicates model generalization
BLEU Score	1.034	N-gram precision-based evaluation result

```
Question:  How do I set up a web server using Nginx?
Maximum Reply Length:  30
i dont know how to set up a server
```

Fig. 6: Greedy response generator after cleantext

VII. Conclusion

This project presented the development of a Transformer-based chatbot trained on the Ubuntu Dialogue Corpus for technical support applications. The system was designed using an encoder-decoder architecture with multi-head self-attention and sinusoidal positional encodings, allowing it to process and generate coherent multi-turn dialogues in a non-recurrent, fully parallelized manner.

A comprehensive data preprocessing pipeline was implemented to clean and structure the dialogue corpus, resulting in over 1.8 million high-quality multi-turn conversations. Subword tokenization using SentencePiece ensured vocabulary flexibility and robust handling of rare or technical terms.

The final Transformer model was trained over 200,000 batches, demonstrating steady convergence with a reduction in training loss from **4.707** to **4.236**, and a validation loss of **4.027**. The BLEU score achieved was **1.034**, reflecting early-stage model capability in handling open-domain queries. Although the BLEU score appears low, it is important to note that open-ended dialogue evaluation remains a challenging task, and n-gram precision alone does not fully capture semantic relevance or context awareness.

Qualitative analysis showed that the model could produce relevant, grammatically correct, and context-sensitive responses, outperforming legacy GRU-based sequence models and shallow Transformer variants in both accuracy and fluency.

Future Work:

- **Extended Training:** Continued training over more epochs and larger subsets of the corpus is expected to improve BLEU scores and overall performance.
- **Beam Search Decoding:** Replacing greedy decoding with beam search could improve response diversity and completeness.
- **Reinforcement Learning (RLHF):** Integrating reward-based optimization (e.g., PPO or user feedback) can help the chatbot align better with desired behaviors and improve long-term coherence.
- **Evaluation Enhancements:** Incorporating embedding-based metrics like BERTScore or human evaluation may provide more accurate assessments of response quality.
- **Domain Expansion:** While this model is tailored for Ubuntu support, the same architecture and training pipeline can be adapted to other technical domains or general-purpose chatbots.

In conclusion, this project lays a strong foundation for scalable, context-aware conversational agents using modern Transformer architectures. With further tuning and evaluation, the model can serve as a deployable solution for real-time technical support systems.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL*, 2019.

[3] A. Radford et al., "Improving language understanding by generative pre-training," *OpenAI*, Tech. Rep., 2018.

[4] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[5] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

The project was completed collaboratively with the following jointly completed tasks:

- Discussed and finalized the project topic.
- Selected the dataset together.
- Developed the project approach collectively.

The individual contributions are detailed below:

- **Manoj Kumar Surabhi** Preprocessing the code and preparation of the presentation slides.
- **Sumanth Reddy Thandra** Responsible for code preprocessing data and development of code and Report.
- **Rahul Sajith**